NILEBRIDGE PROFESSIONAL ACADEMY

Flutter Mobile Application Development

DEMONSTRATION CATALOGUE · 2026–2027 · CATALOGUE CODE FL

CATALOGUE OVERVIEW

A hands-on catalogue for building cross-platform mobile applications with Dart and Flutter, integrating APIs, managing state, testing user journeys, and preparing repeatable release workflows.

ACADEMY AND DATA NOTICE

NILEBRIDGE PROFESSIONAL ACADEMY is a fictional demonstration institution. All programme records, durations, curriculum details, assessments, and career descriptions are synthetic examples created for RAG ingestion and retrieval testing. They are not real academy offerings, verified schedules, official accreditation claims, or employment guarantees.

Programme Directory

The programme records below use stable programme identifiers to support retrieval and metadata validation.

Programme ID	Programme title	Level	Duration
FL-DART-101	Dart & Flutter Foundations	Foundation	60 guided learning hours
FL-STATE-201	Flutter State Management & API Integration	Intermediate	80 guided learning hours

Information included for each programme

- Programme identity, level, duration, delivery format, and intended audience
- Entry requirements and programme purpose
- Learning outcomes and week-by-week curriculum
- Practical labs, assessment weights, and grading criteria
- Capstone brief, deliverables, tools, and FAQ
- Provenance and accreditation status

PROGRAMME 01 · FL-DART-101

Dart & Flutter Foundations

Indicative hours	Duration	Level	Delivery
60 guided learning hours	8 weeks	Foundation	Guided coding workshops and app-building labs.

Programme summary

Introduces Dart and Flutter's widget model, including layouts, navigation, forms, themes, debugging, and project organization.

Intended audience

Beginners interested in mobile app development.

Entry requirements

Basic programming concepts are helpful but not mandatory.

Learning outcomes

- Use Dart variables, functions, classes, collections, and null safety.
- Build screens using reusable Flutter widgets and layout constraints.
- Implement navigation and validated user forms.
- Run and debug an app in an emulator or supported device environment.

Week-by-week curriculum

Week	Topic	Coverage	Practice / evidence
1	Dart setup and syntax	Variables, types, expressions, functions, and editor setup.	Write Dart exercises.
2	Collections and classes	Lists, maps, classes, null safety, and error handling.	Model sample application records.
3	Async basics	Understand Future and asynchronous I/O concepts.	Write a small asynchronous example.
4	Widget composition	Widget tree, layouts, constraints, and reusable components.	Build a home screen.
5	Navigation	Routes, screen composition, and passing simple values.	Build a multi-screen flow.
6	Forms and validation	Input, validation rules, feedback, and form state.	Build a validated form.
7	Themes and debugging	Reusable styling, assets, emulator use, and debugging.	Polish the app UI.
8	Capstone	Integrate screens and document behavior.	Demonstrate the app.

Practical labs

- Dart language exercises
- Widget layout practice
- Navigation flow
- Form validation

Assessment structure

Component	Weight	Evidence / criteria
Dart exercises	25%	Correct language use and readable code.
UI labs	35%	Layout behavior and widget composition.
Capstone	40%	Working app, navigation, validation, and documentation.

Capstone project

Build a multi-screen Flutter app with consistent theming, navigation, validated input, and setup instructions.

Suggested grading rubric

- Dart fundamentals: 20%
- UI and layout: 30%

- Interaction behavior: 25%
- Testing and documentation: 25%

Tools and technologies

Dart · Flutter SDK · Android Studio or VS Code · emulator · Git

Frequently asked questions

What is a widget?

A building block in Flutter's UI composition model.

Do I need a physical phone?

An emulator may be sufficient for course exercises, depending on the task and machine setup.

Is this a backend course?

No. Backend and API integration are covered in a separate programme.

Record and provenance

Department: Mobile Development | Skill level: Foundation | Accreditation: not verified | Country: not specified | Source status: synthetic demonstration data | Catalogue: Flutter Mobile Application Development | Programme: Dart & Flutter Foundations | Source file: 04_Flutter_Expanded_Catalogue.pdf.

PROGRAMME 02 · FL-STATE-201

Flutter State Management & API Integration

Indicative hours	Duration	Level	Delivery
80 guided learning hours	8 weeks	Intermediate	Project-based labs with code review.

Programme summary

Builds an API-driven application with explicit loading, success, empty, and error states; organized data access; and tests for key behaviors.

Intended audience

Learners able to build Flutter screens and use Dart asynchronous features.

Entry requirements

Dart basics, widget composition, navigation, and introductory JSON concepts.

Learning outcomes

- Choose and document a state-management approach for a small app.
- Call REST APIs and interpret response and error conditions.
- Convert JSON into typed application models.
- Separate UI, repository, and service responsibilities.
- Test data parsing and key UI state transitions.

Week-by-week curriculum

Week	Topic	Coverage	Practice / evidence
1	State and UI responsibilities	Stateful widgets, state lifting, and separation of concerns.	Map state changes for a screen.
2	State management choice	Compare a selected approach such as Provider or Riverpod for the project.	Implement one consistent state approach.
3	HTTP and REST	Requests, response codes, timeouts, and connectivity failures.	Call a sample API.
4	JSON and models	Parsing, optional fields, and invalid payloads.	Build typed data models.
5	Loading and errors	Represent loading, empty, success, and failure states.	Create a resilient data screen.
6	Repository pattern	Separate presentation and network code.	Refactor data access.
7	Testing	Test model parsing, repository behavior, and widget states.	Write focused tests.
8	Capstone	Integrate the app and explain architecture decisions.	Demo and submit tests.

Practical labs

- API request exercise
- JSON parsing
- Error-state UI
- Repository and widget tests

Assessment structure

Component	Weight	Evidence / criteria
API labs	25%	Correct request handling and response parsing.
Architecture task	25%	Clear separation and documented state choice.
Capstone and tests	50%	Reliable states, test coverage, and demo.

Capstone project

Create a mobile app that consumes a documented API, shows loading/empty/error states, uses a consistent state-management approach, and includes tests.

Suggested grading rubric

- API and parsing: 25%
- State handling: 25%
- Architecture: 20%
- Testing and documentation: 30%

Tools and technologies

Flutter · Dart · HTTP client package · REST API · Provider or Riverpod · Flutter test tools

Frequently asked questions

Should the app show a blank screen during loading?

No. It should present a clear loading state and handle empty or failed responses.

Can the UI call HTTP requests directly everywhere?

A dedicated service or repository layer usually improves testability and separation of concerns.

Does a successful HTTP response guarantee valid data?

No. The response body still needs validation and safe parsing.

Record and provenance

Department: Mobile Development | Skill level: Intermediate | Accreditation: not verified | Country: not specified | Source status: synthetic demonstration data | Catalogue: Flutter Mobile Application Development | Programme: Flutter State Management & API Integration | Source file: 04_Flutter_Expanded_Catalogue.pdf.